SMART TRIPO-AN INTELLIGENT TRIP PLANNER WEB APPLICATION

Project Submitted to the

BHARATHIDASAN UNIVERSITY, TIRUCHIRAPALL1-24

In partial fulfilment of the requirements for the award of

BACHELOR DEGREE IN COMPUTER APPLICATIONS

Submitted by

NAME	REGISTER NUMBER
SASMITHA S	23050410500112066
YAZHINI R	23050410500112091
YUVASHRI D	23050410500112092

Under the guidance

Dr.S.RETHINAVALLI M.C.A.,M.Phil.,Ph.D., ACADEMIC DIRECTOR

DEPARTMENT OF COMPUTER APPLICATIONS

(AICTE APPROVED)

SHRIMATI INDIRA GANDHI COLLEGE

(Affiliated to Bharathidasan University)

Nationally Accredited at "A" Grade (4th Cycle) by NAAC

An ISO 9001:2015 Certified Institution

TIRUCHIRAPPALLI - 620 002

APRIL 2026

CERTIFICATE

Dr.S.RETHINAVALLI M.C.A.,M.Phil.,Ph.D.,
ACADEMIC DIRECTOR
Department of Computer Applications(AICTE APPROVED),
Shrimati Indira Gandhi College,
Tiruchirappalli-620002.

CERTIFICATE

This is to certify that the project work entitled

SMART TRIPO-AN INTELLIGENT TRIP PLANNER WEB APPLICATION

Submitted by

NAME	REGISTER NUMBER
SASMITHA S	**23050410500112066**
YAZHINI R	**23050410500112091**
YUVASHRI D	**23050410500112092**

Is a bonafide record of the work done by the above mentioned candidates in partial fulfilment of the requirements for the award of the degree of **"BACHELOR OF COMPUTER APPLICATIONS "** and has been carried out under my guidance during the Sixth semester of the academic year (2025-2026).

Signature of the Academic Director Signature of the Guide

Viva-Voce Examination

The Project Viva Voce Examination was held on ----------

Signature of Internal Examiner

Signature of External Examiner

ACKNOWLEDGEMENT

ACKNOWLEDGEMENT

First and foremost, we express our praise and thanks to Lord Almighty for his manifold blessing showered on right from admission and till the completion of the project work successfully in the temple of learning.

We endow our grateful thanks and deep sense of gratitude to our respected **PRINCIPAL Dr. P. GAJALAKSHMI, M.Sc., M.Phil. Ph.D.**, Shrimati Indira Gandhi College, Tiruchirappalli and the management for providing us all the Support carry out the project.

We express our thanks and deep sense of gratitude to **our ACADEMIC DIRECTOR Dr. S. RETHINAVALLI, MCA., M.Phil., Ph.D.**, of Computer Applications, Shrimati Indira Gandhi College, Tiruchirappalli for sustained Encouragement shown towards the successful completion of this work.

We take immense pleasure in expressing our heartfelt gratitude to our guide **Dr. S. RETHINAVALLI, MCA., M.Phil., Ph.D.**, Academic Director, Department of Computer Applications, for her guidance and encouragement during the course of our study and in successfully accomplishing this project.

We also submitted our heartfelt thanks to our tutor **Ms. J. RATHNA B.E., M.Tech.,** Assistant Professor, Department of Computer Applications, Shrimati Indira Gandhi College Tiruchirappalli for their motivation and support rendered during the tenure of our course.

CONTENT

S.NO	TITLE	PAGENO
	ABSTRACT	
1	**INTRODUCTION**	1
2	**SYSTEM ANALYSIS** 2.1 Existing System 2.2 Proposed System	2
3	**SYSTEM CONFIGURATION** 3.1 Hardware Specification 3.2 Software Specification	3
4	**PROJECT DESCRIPTION** 4.1 Software Description 4.2 Modules Description	4
5	**SYSTEM DESIGN** 5.1 Use Case Diagram 5.2 Data Flow Diagram 5.3 Sequence Diagram	6
6	**TESTING** 6.1 Unit Testing 6.2 System Testing 6.3 Integration Testing	12
7	**SOFTWARE IMPLEMENTATION** 7.1 Source Code 7.2 Screen Layouts	16
	CONCLUSION	29
	FUTURE ENHANCEMENTS	30
	BIBLIOGRAPHY	31

ABSTRACT

Smart Tripo is an intelligent web-based trip planning application developed to simplify and enhance the travel planning experience for users. In today's fast-paced world, planning a trip involves searching for destinations, comparing travel options, checking budgets, organizing schedules, and managing accommodations. This process can be time-consuming and confusing. Smart Tripo-An intelligence trip planner web application aims to solve this problem by providing an automated, user-friendly, and personalized travel planning system using modern web technologies.

The application is primarily developed using Python as the backend programming language to handle business logic, data processing, and intelligent recommendations. Frameworks such as Flask or Django can be used to build a secure and scalable web server. The frontend is designed using HTML for structure and CSS for styling, ensuring a responsive, interactive, and visually appealing user interface.

Smart Tripo-An intelligence trip planner web application allows users to input their travel preferences such as destination, budget range, number of days, interests (adventure, nature, historical places, etc.), and travel dates. Based on this input, the system intelligently generates a customized travel itinerary. The application can integrate features like route optimization, cost estimation, hotel suggestions, nearby attractions, and daily scheduling. The intelligent component of the system can use basic recommendation algorithms or rule-based logic to suggest the most suitable travel plans according to user preferences.

The system also includes user authentication features where users can register, log in, and save their planned trips for future reference. An admin panel can be implemented to manage destination data, update travel packages, and monitor user activity. The application ensures efficient data storage and retrieval using a database such as MySQL or SQLite.

Smart Tripo-An intelligence trip planner web application focuses on improving user convenience, reducing manual effort, and delivering personalized travel experiences. It is especially useful for students, families, and working professionals who want quick and reliable travel planning assistance. The project demonstrates the integration of backend intelligence with frontend design, making it a complete full-stack web application.

1.INTRODUCTION

In the modern digital era, travel and tourism have become an essential part of people's lives. Whether for relaxation, education, business, or adventure, individuals frequently plan trips to different destinations. However, organizing a trip is often a complex and time-consuming process. It involves searching for destinations, comparing transportation options, selecting accommodations, planning daily schedules, estimating budgets, and ensuring everything fits within time constraints. Many travelers depend on multiple websites and manual planning methods, which can lead to confusion, poor time management, and increased expenses.

To address these challenges, Smart Tripo – An Intelligent Trip Planner Web Application is developed as a comprehensive solution for efficient and personalized travel planning. Smart Tripo is a web-based application designed to simplify the entire trip planning process by providing intelligent suggestions and automated itinerary generation based on user preferences. The application combines backend logic developed using Python with frontend technologies such as HTML and CSS to create an interactive and user-friendly platform.

The main objective of Smart Tripo-An intelligence trip planner web application is to provide users with a smart system that reduces manual effort and enhances travel decision-making. By collecting inputs such as destination, number of travel days, budget, type of trip (family, solo, adventure, etc.), and personal interests, the system processes this data and generates a structured travel plan. The application can recommend attractions, suggest accommodations, estimate costs, and create a day-wise itinerary. This intelligent recommendation approach ensures that the travel plan is customized and aligned with the user's needs.

The backend of the system, developed using Python, manages core functionalities such as data processing, itinerary generation, user authentication, and database interactions. Frameworks like Flask or Django can be used to handle server-side operations. The frontend, built with HTML and CSS, ensures a responsive design that allows users to easily navigate through features, input preferences, and view generated plans. The system can also use a database such as MySQL or SQLite to store user data, travel details, and destination information securely.

Smart Tripo-An intelligence trip planner web application not only benefits individual travelers but also demonstrates practical application of full-stack web development concepts. It integrates programming logic, database management, and user interface design into a single system. The project showcases how intelligent systems can improve traditional processes by making them faster, more accurate, and user-centric.

2.SYSTEM ANALYSIS

2.1 EXISTING SYSTEM

- Users must manually search for places, compare prices, and create their own travel schedules.
- Travelers need to use different websites for booking hotels, transport, and sightseeing information.
- Most platforms provide general recommendations rather than customized itineraries based on user preferences.
- Searching, comparing, and organizing trip details takes significant time and effort.
- Users have to prepare day-wise plans themselves without system assistance.
- Budget estimation and cost management are usually done manually.
- Traditional agencies provide packages but may not always offer flexible or user-specific plans.

2.2 PROPOSED SYSTEM

- Combines all travel planning activities in a single web application.
- Automatically creates a day-wise travel plan based on user inputs.
- Suggests destinations and attractions according to user interests.
- Generates travel plans within the user's specified budget.
- Allows users to securely create accounts and manage their trips.
- Handles data processing, logic implementation, and system functionality.
- Stores user information, destination details, and saved trip plans.
- Reduces manual effort and speeds up the planning process.
- Provides an easy-to-use and responsive web design.
- Enables management of destinations, user data, and system updates.

3.SYSTEM CONFIGURATION

3.1HARDWARE SPECIFICATION

Processor	: Intel Core i3 or above (or equivalent AMD processor)
RAM	: Minimum 4 GB RAM (8 GB recommended for better performance)
Hard Disk	: 250 GB or above (SSD recommended for faster performance)
Monitor	: 15-inch or higher display for better development experience
Input Devices	: Keyboard and Mouse
Internet Connection	: Required for testing web application and installing dependencies

3.2SOFTWARE SPECIFICATION

Operating System	: Windows 10/11, Linux, or macOS
Backend	: Python (for handling business logic and server-side processing)
Web Framework	: Flask or Django (for building the web application)
Frontend	: HTML (for webpage structure), CSS (for styling and responsive design)
Database	: MySQL or SQLite (for storing user and travel data)
Web Browser	: Google Chrome, Mozilla Firefox, or Microsoft Edge
Code Editor / IDE	: Visual Studio Code / PyCharm / Sublime Text
Python Libraries	: Flask/Django, SQLite3 / MySQL Connector

4.PROJECT DESCRIPTION

4.1 SOFTWARE DESCRIPTION

- Smart Tripo – An Intelligent Trip Planner Web Application is a web-based software system developed to simplify and automate the process of travel planning. The application is designed using Python as the backend programming language and HTML and CSS for the frontend interface. The system provides users with an intelligent and personalized platform to plan trips efficiently.
- The main purpose of this software is to reduce the complexity involved in manual trip planning. Instead of using multiple websites to search for destinations, accommodations, budgets, and schedules, Smart Tripo integrates all these features into a single platform. The system collects user inputs such as destination, number of travel days, budget range, and interests. Based on these inputs, the backend processes the data and generates a customized, day-wise travel itinerary.
- The software follows a client-server architecture. The frontend, developed using HTML and CSS, provides a responsive and user-friendly interface where users can interact with the system. The backend, built using Python with frameworks such as Flask or Django, handles business logic, user authentication, data processing, and itinerary generation. A database such as MySQL or SQLite is used to store user information, travel data, and destination details securely.
- The application consists of different modules including:
    - ✓ User Registration and Login Module
    - ✓ Travel Preference Input Module
    - ✓ Itinerary Generation Module
    - ✓ Budget Estimation Module
    - ✓ Admin Management Module
    - ✓ Database Management Module
- The software is designed to be scalable, meaning it can be enhanced in the future with advanced features such as AI-based recommendations, real-time booking integration, map integration, and payment gateways.

4.2 MODULES DESCRIPTION

1. User Module

This module allows users to register and log in to the system. Users can enter their travel details such as destination, number of days, budget, and preferences. They can also view and manage their planned trips.

2. Travel Input Module

In this module, users provide trip-related information. The system collects data like travel dates, type of trip (family, solo, adventure), and interests to generate a suitable plan.

3. Itinerary Generation Module

This is the core module of the system. It processes user inputs using Python backend logic and automatically generates a day-wise travel itinerary.

4. Budget Management Module

This module estimates the total cost of the trip based on user budget and suggests suitable travel options within that limit.

5. Database Module

The database module stores user details, destination information, and saved trip plans. It ensures secure storage and easy retrieval of data.

6. Admin Module

The admin module allows the administrator to add, update, or delete destination details and manage user information.

5.SYSTEM DESIGN

5.1 USE CASE DIAGRAM

Actors:

- User
- Admin

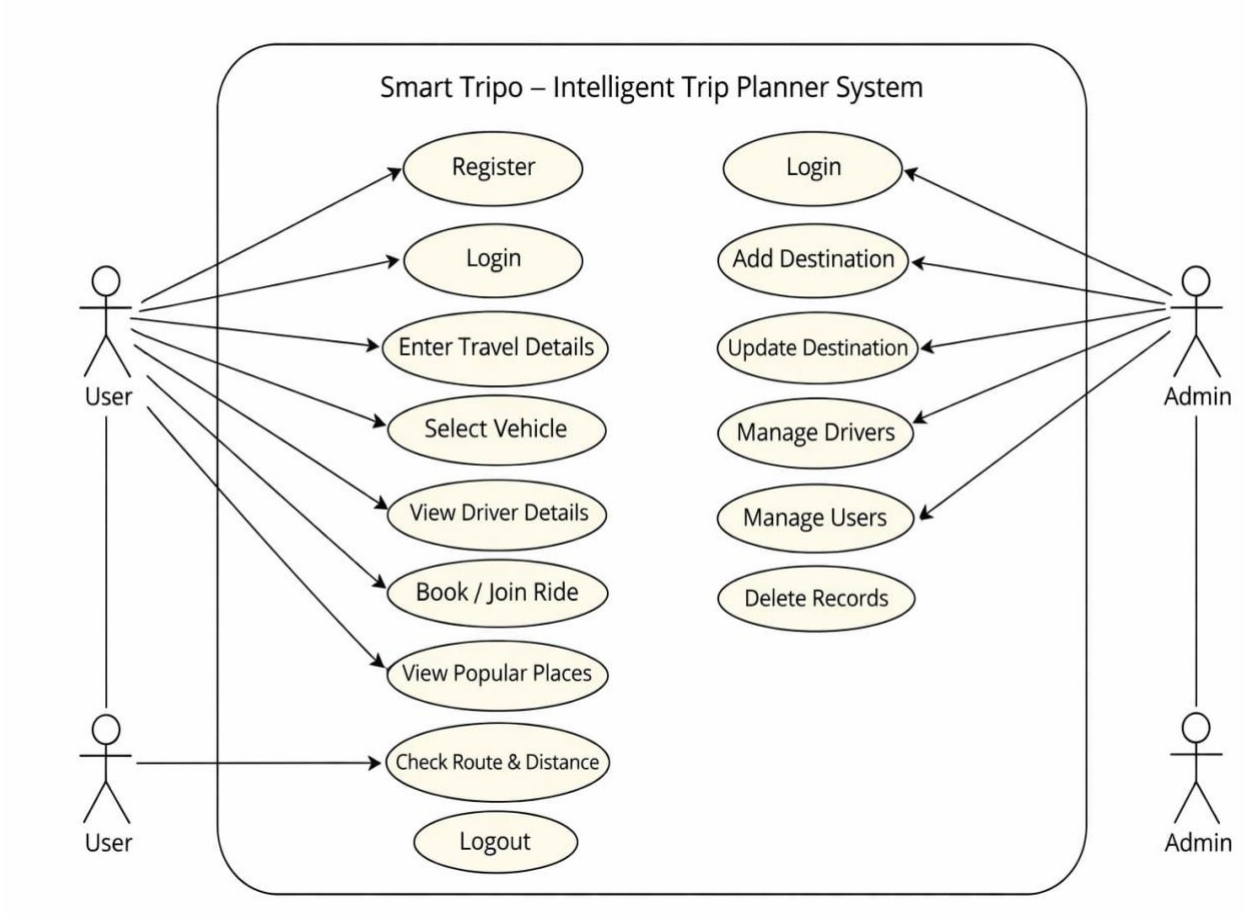
Use Cases for User:

- Register
- Login
- Enter Destination
- Select Travel Date & Time
- Choose Travel Mode (Car/Bike)
- View Available Drivers
- View Driver Details
- Join / Book Ride
- View Popular Places
- Check Route & Distance
- Logout

Use Cases for Admin:

- Login
- Add Destination Details
- Update Destination Information
- Manage Driver Details
- Manage User Accounts
- View System Data
- Delete/Modify Records

Use Case Diagram



5.2 DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) is a graphical representation of how data moves through a system. It shows the flow of information between external entities, processes, and data stores. In Smart Tripo, the DFD explains how user inputs are processed and how travel plans are generated and stored.

Level 0 DFD (Context Diagram)

The Level 0 DFD represents the entire Smart Tripo system as a single process. It shows the interaction between external entities and the system.

External Entities:

- User
- Admin

Process:

- Smart Tripo System

Explanation:

- The User provides travel details such as destination, date, budget, and preferences to the system.
- The system processes this information and generates travel plans, driver details, and booking confirmations.
- The Admin manages destination data, driver details, and user records.
- The system stores and retrieves information from the database.
- The Level 0 DFD gives a high-level overview of how the system interacts with users and administrators.

Level 1 DFD

The Level 1 DFD breaks the system into major functional processes.

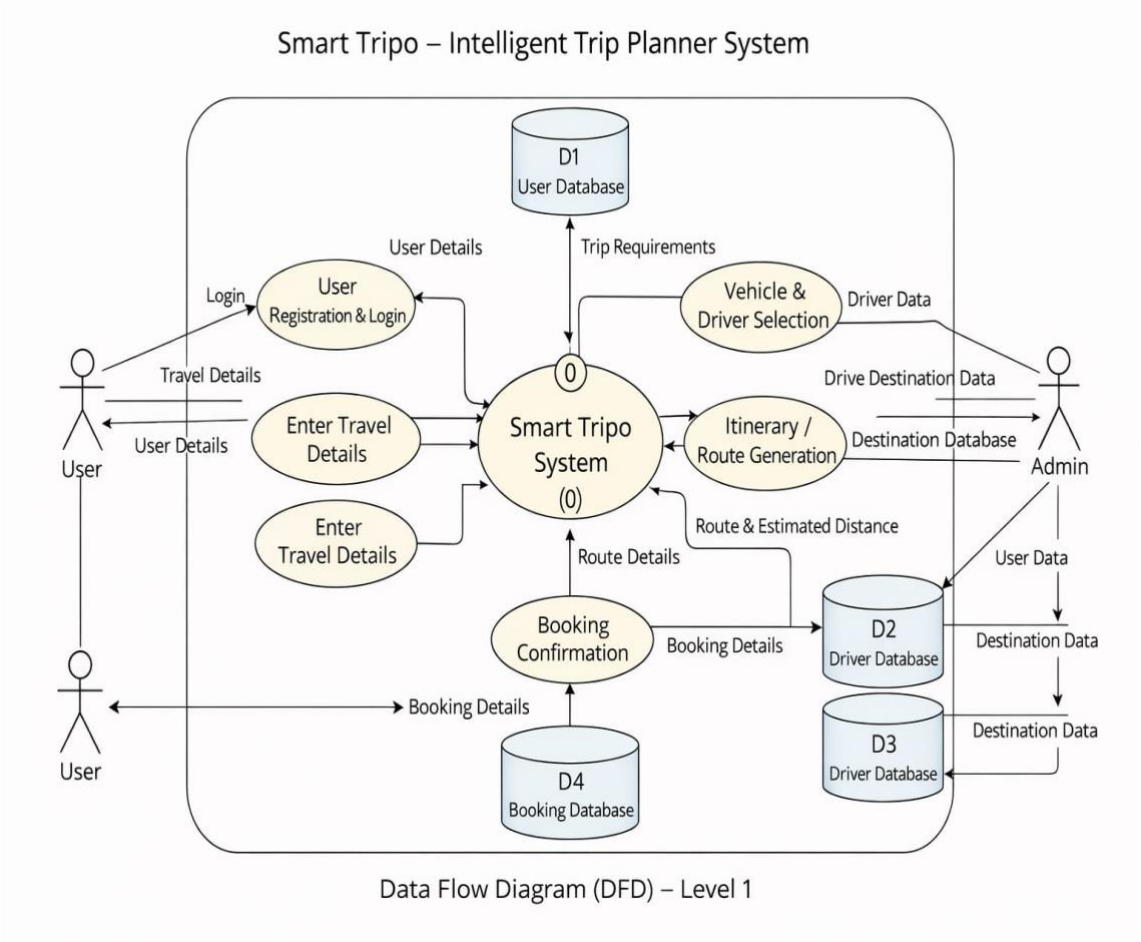
Main Processes:

1. User Registration and Login
    - The user enters login credentials.
    - The system verifies the details using the User Database.
    - If valid, access is granted.
2. Enter Travel Details
    - The user enters destination, travel date, number of seats, and preferences.
    - The system receives and processes this data.
3. Vehicle and Driver Selection
    - The user selects vehicle type (Car/Bike).

- The system retrieves available drivers from the Driver Database.
- Driver details such as rating and experience are displayed

4. Route and Distance Calculation
    - The system checks the Destination Database.
    - It calculates route and estimated distance.
5. Booking Confirmation
    - The user confirms the ride.
    - Booking details are stored in the Booking Database.
6. Admin Management
    - Admin logs in.
    - Admin adds, updates, or deletes destination and driver information.
    - Changes are stored in the database.

Data Flow Diagram



5.3 SEQUENCE DIAGRAM

A Sequence Diagram is a UML diagram that illustrates the interaction between objects in a system arranged in a time sequence. It shows how and in what order the messages are exchanged to complete a specific task.

Key Elements of a Sequence Diagram

Actor:

The user or external entity interacting with the system.

Objects:

Components involved in the process (e.g., Web Interface, System, Database).

Lifeline :

A vertical dashed line representing the life of an object.

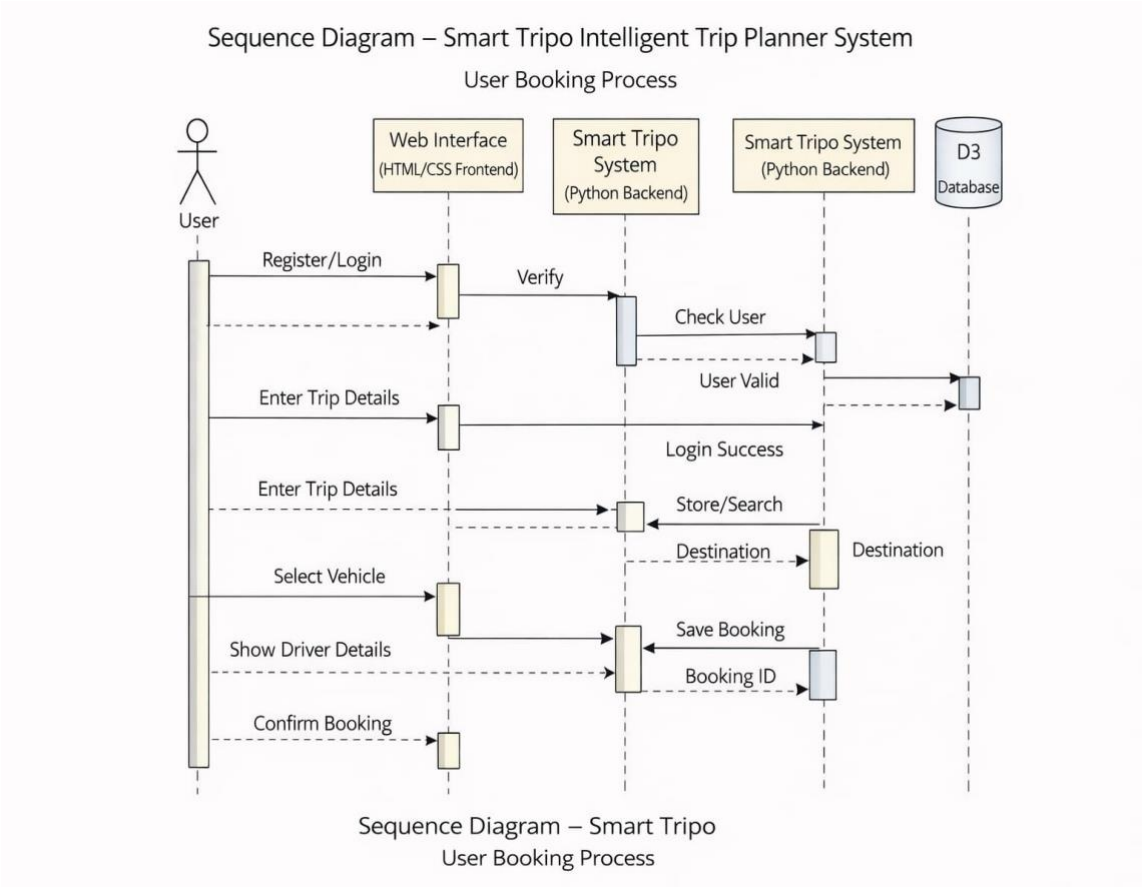
Messages:

Arrows showing communication between objects.

Activation Bar:

Shows the time during which an object is performing an action.

Sequence Diagram



6.TESTING

6.1 UNIT TESTING

Unit testing is a software testing method where individual components or modules of a program are tested separately to ensure they work correctly.

1.Objective of Unit Testing

- To verify each module works as expected
- To detect errors at an early stage
- To ensure correct logic implementation

2.Scope of Unit Testing

- Testing individual functions
- Testing classes or methods
- Testing database queries separately
- Testing backend logic (Python functions in Smart Tripo)

3.Test Cases Design

- Writing test cases for each function
- Checking valid and invalid inputs
- Verifying expected output
- **Example (Smart Tripo):**
    - Test login function with correct username/password
    - Test booking function with missing fields

4.White Box Testing

- Unit testing mainly follows white box testing, where the internal code structure is tested.

5.Test Data Preparation

- Creating sample input data
- Using dummy user accounts
- Creating sample booking details

6.Error Handling Testing

- Checking system response for invalid inputs
- Testing empty fields
- Handling incorrect destination names

7.Automation in Unit Testing

- Using testing tools like unittest in Python
- Writing automated test scripts
- Running test cases automatically

8.Advantages of Unit Testing

- Early bug detection
- Reduces debugging cost
- Improves code quality
- Makes system more reliable

9.Limitations of Unit Testing

- Cannot detect integration errors
- Time-consuming if not automated
- Requires coding knowledge

6.2 SYSTEM TESTING

System Testing is the process of testing the complete and fully integrated software system to verify that it meets the specified requirements. In Smart Tripo – Intelligent Trip Planner Web Application, system testing ensures that all modules such as user login, trip planning, vehicle selection, booking process, and admin management work together correctly as a complete system.

Objectives of System Testing

- To verify the overall functionality of the system
- To ensure all modules are properly integrated
- To check whether the system meets user requirements
- To identify errors before deployment

System Testing Performed in Smart Tripo

1. **Functional Testing**

- The complete system was tested to ensure all features work correctly.
- User registration and login validation

- Entering travel details
- Vehicle and driver selection
- Booking confirmation
- Admin data management

2. **Integration Testing**

- Checked whether all modules work together smoothly.
- **For example:**
- Login module connected properly with the database
- Booking module storing data correctly

3. Validation Testing

- Ensured that the system meets the project requirements and objectives.
- Correct itinerary generation
- Accurate route and distance display

4.Performance Testing

- Tested system response time and loading speed under normal conditions.

5. Security Testing

- Verified secure login, proper authentication, and protected database access.

6.Usability Testing

- Ensured that the interface is simple, user-friendly, and easy to navigate.

6.3 INTEGRATION TESTING

Integration Testing is a software testing method where individual modules of a system are combined and tested together to verify that they interact properly. In Smart Tripo – Intelligent Trip Planner Web Application, integration testing ensures that all modules such as login, trip planning, driver selection, booking, and admin management work together smoothly without errors.

Objective of Integration Testing

- To check data flow between modules
- To identify interface errors
- To ensure proper communication between frontend, backend, and database
- To verify combined functionality of the system

Integration Testing in Smart Tripo

The following module integrations were tested:

1.User Module + Database

- Verified that registration details are stored correctly.
- Checked whether login credentials are validated properly from the database.

2.Travel Input Module + Itinerary Module

- Ensured travel details entered by the user are processed correctly.
- Verified that the system generates appropriate route and driver details.

3.Vehicle Selection Module + Driver Database

- Checked whether selected vehicle type displays correct available drivers.
- Verified driver information retrieval from database.

4.Booking Module + Booking Database

- Ensured booking confirmation details are stored properly.
- Verified that booking ID is generated successfully.

5.**Admin Module + All Databases**

- Checked whether admin updates (add, modify, delete) are reflected in the system.
- Verified that changes are updated in destination and driver databases

7. SOFTWARE IMPLEMENTATION

7.1 SOURCE CODE

```
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.decorators import login_required
from django.contrib.auth.models import User
from django.contrib import messages
from .models import Vehicle, Driver, Trip
from .forms import VehicleForm, DriverForm, TripForm, UserForm


# ================= LOGIN =================

def login_view(request):
    if request.method == "POST":
        username = request.POST.get("username")
        password = request.POST.get("password")

        user = authenticate(request, username=username, password=password)

        if user:
            login(request, user)
            print("Login success")
            return redirect('trip_list')
        else:
            messages.error(request, "Invalid credentials")

    return render(request, 'login.html')
```

```
def logout_view(request):
    logout(request)
    return redirect('login')


# ================= USER CRUD =================

@login_required
def user_list(request):
    query = request.GET.get('search')
    users = User.objects.all()

    if query:
        users = users.filter(username__icontains=query)

    return render(request, 'users.html', {'users': users})


@login_required
def user_create(request):
    form = UserForm(request.POST or None)

    if form.is_valid():
        user = form.save(commit=False)
        user.set_password(form.cleaned_data['password'])
        user.save()
        messages.success(request, "User created successfully")
```

```
        return redirect('user_list')

    return render(request, 'form.html', {'form': form, 'title': 'Add User'})



@login_required
def user_edit(request, pk):
    user = get_object_or_404(User, pk=pk)
    form = UserForm(request.POST or None, instance=user)

    if form.is_valid():
        user = form.save(commit=False)
        user.set_password(form.cleaned_data['password'])
        user.save()
        messages.success(request, "User updated successfully")
        return redirect('user_list')

    return render(request, 'form.html', {'form': form, 'title': 'Edit User'})



@login_required
def user_delete(request, pk):
    user = get_object_or_404(User, pk=pk)

    if request.method == "POST":
        user.delete()
        messages.success(request, "User deleted successfully")
        return redirect('user_list')
```

```
    return render(request, 'confirm_delete.html')


# ================= VEHICLE CRUD =================

@login_required
def vehicle_list(request):
    vehicles = Vehicle.objects.all()
    return render(request, 'vehicles.html', {'vehicles': vehicles})



@login_required
def vehicle_create(request):
    form = VehicleForm(request.POST or None)

    if form.is_valid():
        form.save()
        messages.success(request, "Vehicle added successfully")
        return redirect('vehicle_list')

    return render(request, 'form.html', {'form': form, 'title': 'Add Vehicle'})



@login_required
def vehicle_edit(request, pk):
    vehicle = get_object_or_404(Vehicle, pk=pk)
    form = VehicleForm(request.POST or None, instance=vehicle)

    if form.is_valid():
```

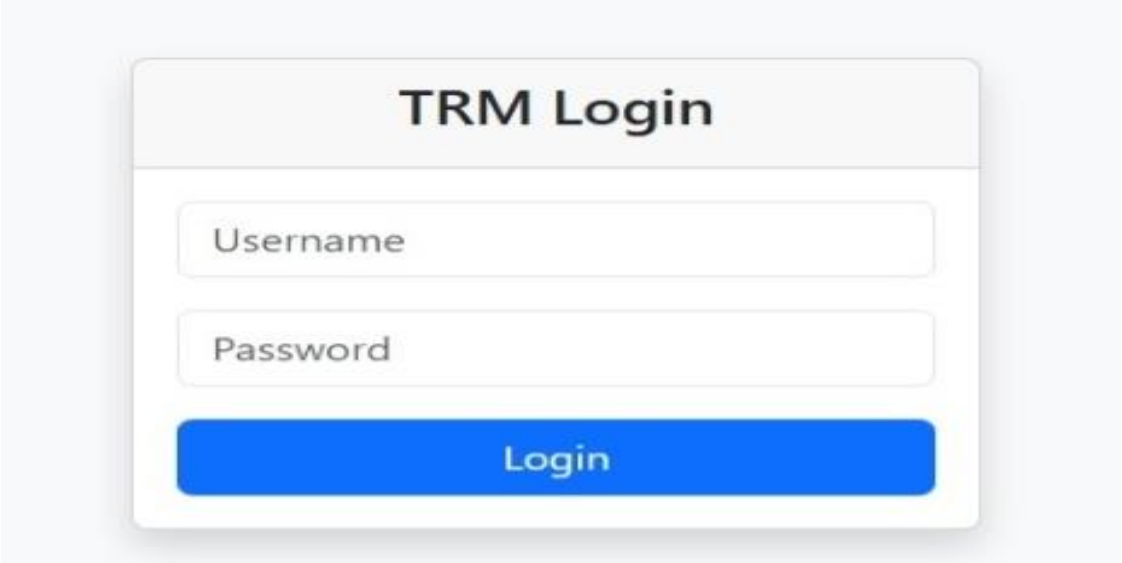
```
        form.save()
        messages.success(request, "Vehicle updated successfully")
        return redirect('vehicle_list')

    return render(request, 'form.html', {'form': form, 'title': 'Edit Vehicle'})



@login_required
def vehicle_delete(request, pk):
    vehicle = get_object_or_404(Vehicle, pk=pk)

    if request.method == "POST":
        vehicle.delete()
        messages.success(request, "Vehicle deleted successfully")
        return redirect('vehicle_list')

    return render(request, 'confirm_delete.html')



# ================= DRIVER CRUD =================

@login_required
def driver_list(request):
    drivers = Driver.objects.all()
    return render(request, 'drivers.html', {'drivers': drivers})



@login_required
def driver_create(request):
```

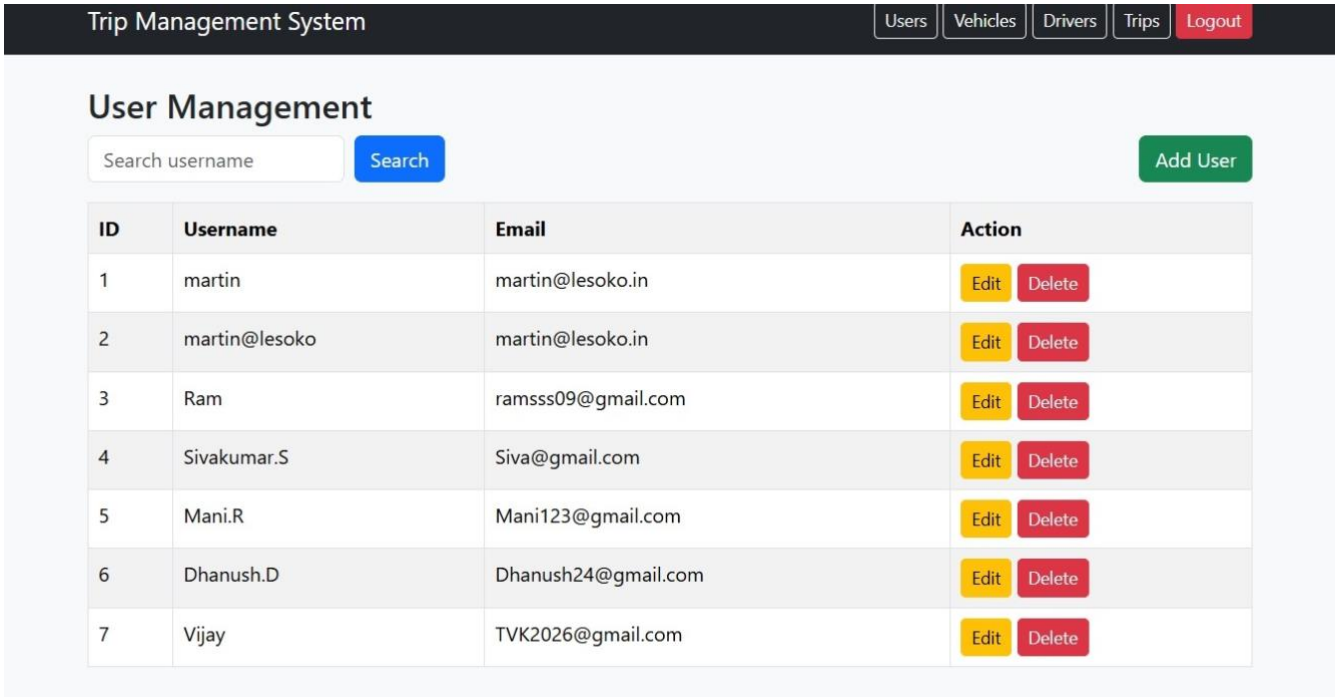
```
    form = DriverForm(request.POST or None)

    if form.is_valid():
        form.save()
        messages.success(request, "Driver added successfully")
        return redirect('driver_list')

    return render(request, 'form.html', {'form': form, 'title': 'Add Driver'})



@login_required
def driver_edit(request, pk):
    driver = get_object_or_404(Driver, pk=pk)
    form = DriverForm(request.POST or None, instance=driver)

    if form.is_valid():
        form.save()
        messages.success(request, "Driver updated successfully")
        return redirect('driver_list')

    return render(request, 'form.html', {'form': form, 'title': 'Edit Driver'})



@login_required
def driver_delete(request, pk):
    driver = get_object_or_404(Driver, pk=pk)

    if request.method == "POST":
        driver.delete()
```

```
        messages.success(request, "Driver deleted successfully")
        return redirect('driver_list')

    return render(request, 'confirm_delete.html')


# ================= TRIP CRUD =================

@login_required
def trip_list(request):
    trips = Trip.objects.select_related('vehicle', 'driver')
    return render(request, 'trips.html', {'trips': trips})


@login_required
def trip_create(request):
    form = TripForm(request.POST or None)

    if form.is_valid():
        form.save()
        messages.success(request, "Trip created successfully")
        return redirect('trip_list')

    return render(request, 'form.html', {'form': form, 'title': 'Add Trip'})


@login_required
def trip_edit(request, pk):
    trip = get_object_or_404(Trip, pk=pk)
```

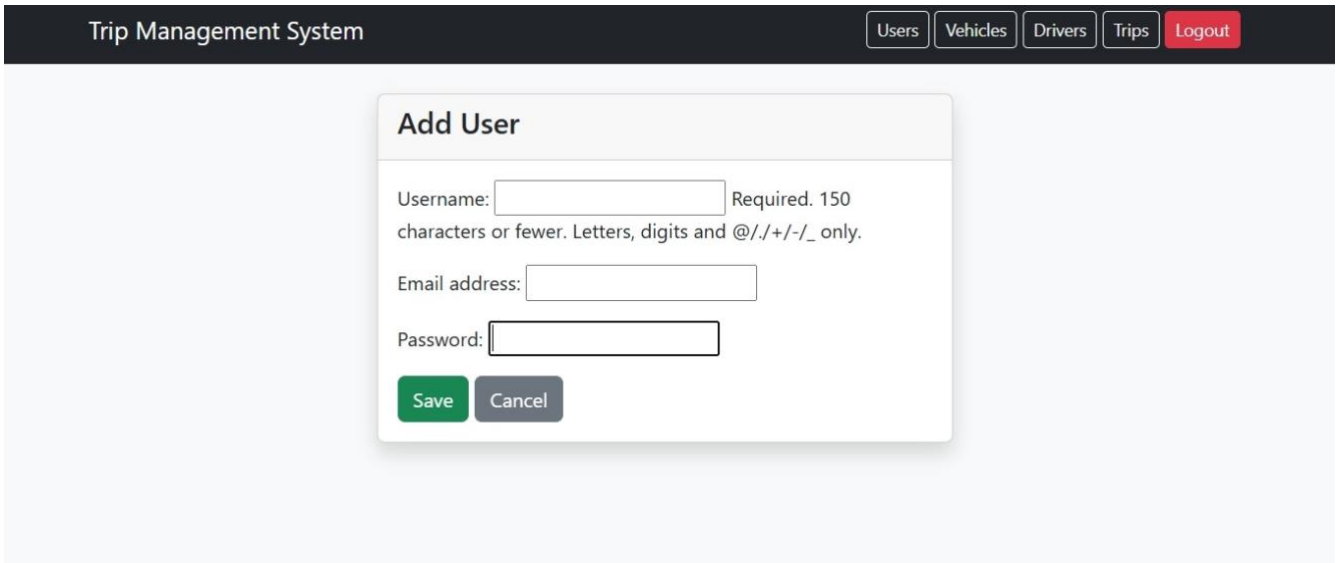
```
    form = TripForm(request.POST or None, instance=trip)

    if form.is_valid():
        form.save()
        messages.success(request, "Trip updated successfully")
        return redirect('trip_list')

    return render(request, 'form.html', {'form': form, 'title': 'Edit Trip'})



@login_required
def trip_delete(request, pk):
    trip = get_object_or_404(Trip, pk=pk)

    if request.method == "POST":
        trip.delete()
        messages.success(request, "Trip deleted successfully")
        return redirect('trip_list')

    return render(request, 'confirm_delete.html')
```

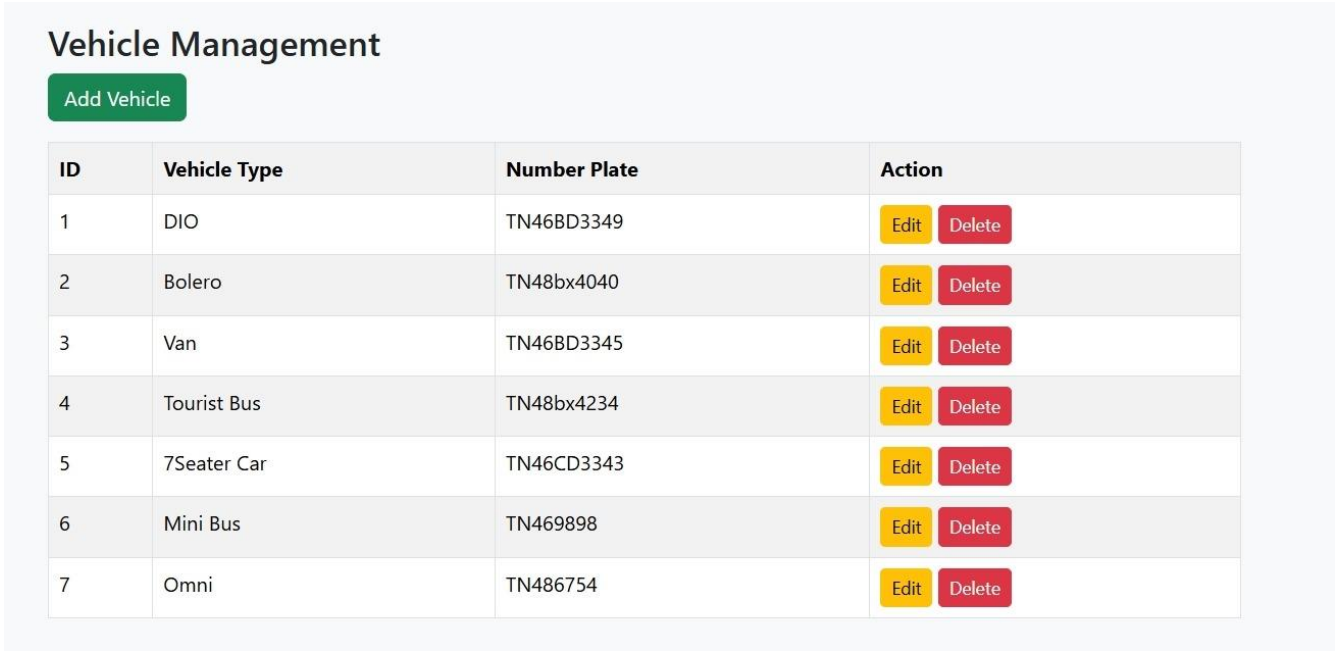
7.2 SCREEN LAYOUTS



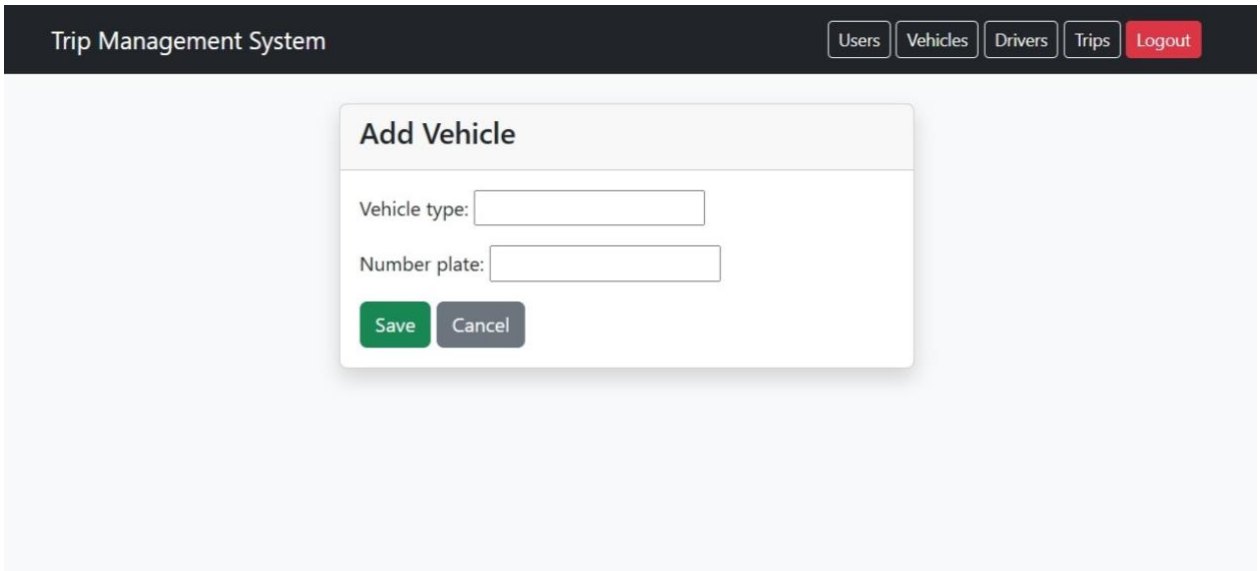


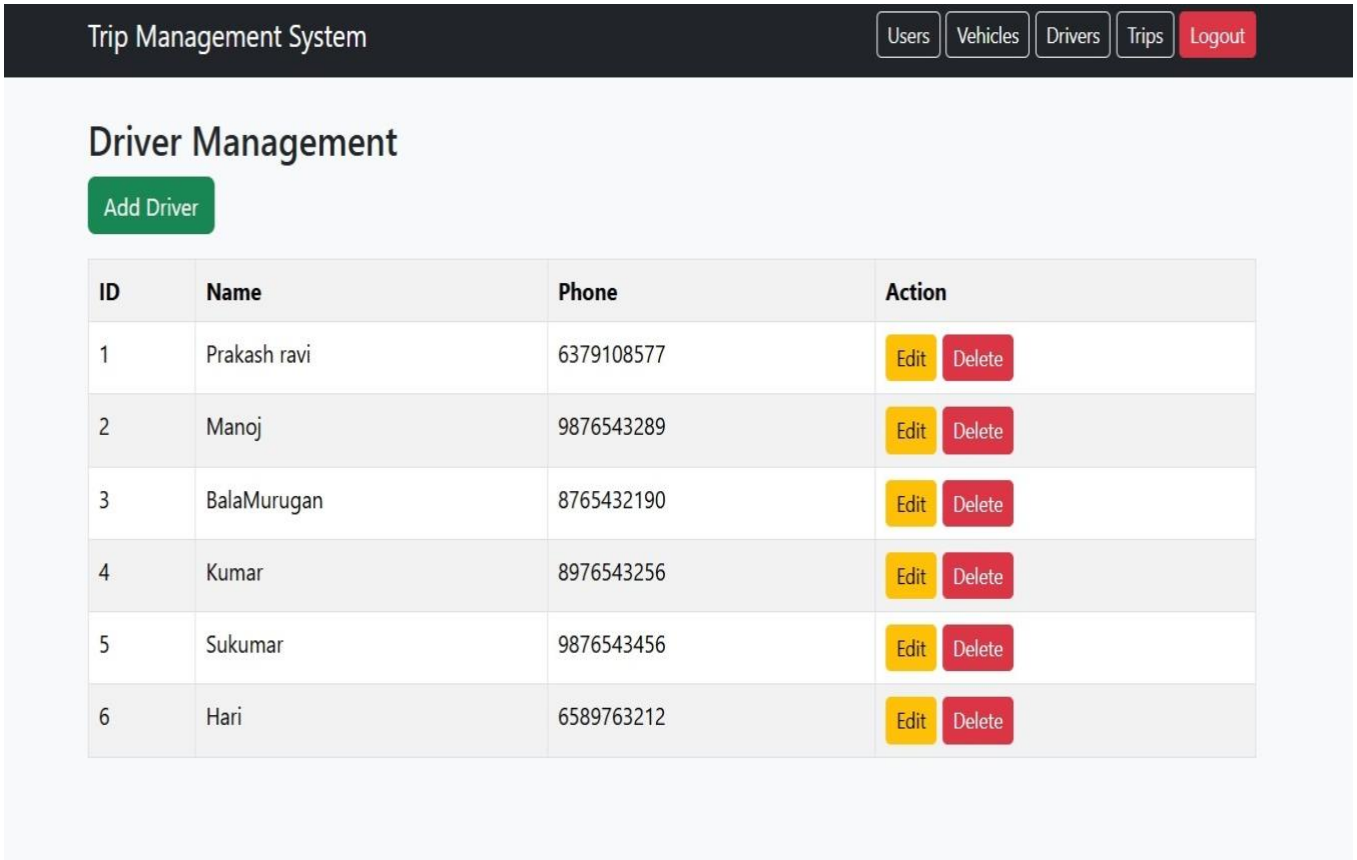
ID	Username	Email	Action
1	martin	martin@lesoko.in	Edit Delete
2	martin@lesoko	martin@lesoko.in	Edit Delete
3	Ram	ramsss09@gmail.com	Edit Delete
4	Sivakumar.S	Siva@gmail.com	Edit Delete
5	Mani.R	Mani123@gmail.com	Edit Delete
6	Dhanush.D	Dhanush24@gmail.com	Edit Delete
7	Vijay	TVK2026@gmail.com	Edit Delete





ID	Vehicle Type	Number Plate	Action
1	DIO	TN46BD3349	Edit Delete
2	Bolero	TN48bx4040	Edit Delete
3	Van	TN46BD3345	Edit Delete
4	Tourist Bus	TN48bx4234	Edit Delete
5	7Seater Car	TN46CD3343	Edit Delete
6	Mini Bus	TN469898	Edit Delete
7	Omni	TN486754	Edit Delete


Trip Management System
Users
Vehicles
Drivers
Trips
Logout
Add Vehicle
Vehicle type:
Number plate:
Save
Cancel


Trip Management System
Users
Vehicles
Drivers
Trips
Logout
Driver Management
Add Driver
ID
Name
Phone
Action
1
Prakash ravi
6379108577
Edit
Delete
2
Manoj
9876543289
Edit
Delete
3
BalaMurugan
8765432190
Edit
Delete
4
Kumar
8976543256
Edit
Delete
5
Sukumar
9876543456
Edit
Delete
6
Hari
6589763212
Edit
Delete

Add Driver

Name:

Phone:

Save Cancel

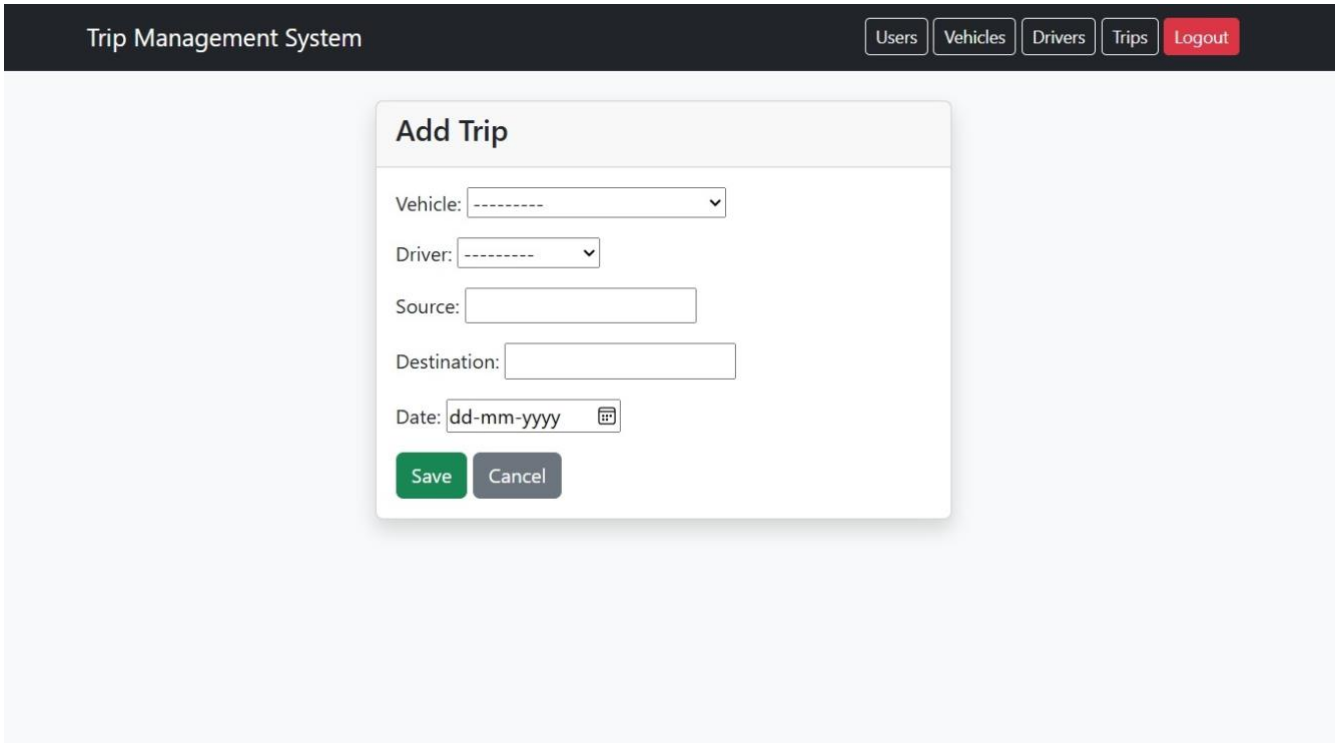
Trip Management System

Users Vehicles Drivers Trips Logout

TRM - Trip Management

Add Trip

ID	Vehicle	Driver	Source	Destination	Date	Action
1	TN46BD3349 (DIO)	Prakash ravi	Trichy	Manappari	Feb. 19, 2026	Edit Delete
2	TN46CD3343 (7Seater Car)	Kumar	Thanjavur	Ooty	March 28, 2026	Edit Delete
3	TN48bx4040 (Bolero)	Manoj	Trichy	Coimbatore	March 10, 0026	Edit Delete
4	TN48bx4234 (Tourist Bus)	Sukumar	Madurai	Varkala	April 14, 2026	Edit Delete
5	TN46BD3349 (DIO)	BalaMurugan	Trichy	Karur	May 16, 2026	Edit Delete


Trip Management System
Users
Vehicles
Drivers
Trips
Logout
Add Trip
Vehicle: ---------
Driver: ---------
Source:
Destination:
Date: dd-mm-yyyy
Save
Cancel

CONCLUSION

The Smart Tripo – Intelligent Trip Planner Web Application was successfully designed and developed to provide users with a simple, efficient, and intelligent travel planning solution. The system helps users plan trips easily by selecting destinations, checking routes, choosing vehicles, and booking rides through a user-friendly web interface built using HTML and CSS, with Python handling the backend operations.

The application simplifies the traditional trip planning process by automating route management, driver selection, and booking confirmation. It reduces manual effort, saves time, and improves accuracy in managing travel details. The admin module ensures proper management of destinations, drivers, and user records, making the system reliable and organized.

Throughout the development process, proper system analysis, design diagrams (DFD, UML, Sequence Diagram), and testing techniques were implemented to ensure system efficiency and accuracy.

In conclusion, Smart Tripo provides a modern and intelligent approach to travel planning and demonstrates how web technologies and backend programming can be integrated to create a practical and user-friendly application. This project enhances both technical knowledge and real-world problem-solving skills.

FUTURE ENHANCEMENT

In the future, Smart Tripo can be enhanced with several advanced features to improve its functionality and user experience. The system can be developed as a mobile application for Android and iOS platforms to increase accessibility and convenience for users. Real-time GPS tracking can be integrated to allow users to monitor their ride location and estimated arrival time. Online payment options such as UPI, credit/debit cards, and net banking can be added to make the booking process more secure and convenient. The application can also include an AI-based recommendation system to suggest destinations based on user preferences and previous travel history. Integrating live weather forecasting will help users plan their trips more effectively. Multi-language support can be implemented to make the system user-friendly for a wider audience. Additionally, a rating and feedback system can be introduced to improve service quality, along with advanced security features such as OTP verification and encrypted data storage. In the long term, Smart Tripo can be expanded to include hotel reservations and flight bookings, transforming it into a complete and intelligent travel management platform.

BIBLIOGRAPHY

BOOK REFERENCES

1. Pressman, R. S., Software Engineering: A Practitioner's Approach, McGraw-Hill Education.

2. Sommerville, Ian, Software Engineering, Pearson Education.

3. Lutz, Mark, Learning Python, O'Reilly Media.

4. Grinberg, Miguel, Flask Web Development, O'Reilly Media.

5. Duckett, Jon, HTML and CSS: Design and Build Websites, Wiley Publications.

WEB REFERENCES

1. W3Schools – Python Tutorial-https://www.w3schools.com

2. Python Official Documentation-https://docs.python.org

3. Mozilla Developer Network (MDN) – HTML & CSS Documentation-https://developer.mozilla.org

4. GeeksforGeeks – Python and Web Development Tutorials-https://www.geeksforgeeks.org

We express our sincere thanks and deep sense of gratitude to our esteemed guide,
Dr. S. Rethinavalli, MCA., M.Phil., Ph.D., Academic Director, Department of Computer Applications, for her invaluable guidance, constant encouragement, and thoughtful support throughout the course of our study. Her insightful suggestions and motivation greatly contributed to the successful completion of this project.

We take immense pleasure in expressing our heartfelt gratitude to our respected guide, **Dr. S. Rethinavalli**, MCA., M.Phil., Ph.D., Academic Director, Department of Computer Applications, for her continuous support, expert guidance, and valuable suggestions throughout our study. Her encouragement and constructive feedback played a vital role in enabling us to complete this project successfully.

We take immense pleasure in expressing our heartfelt sense of gratitude to our guide **Dr. S. RETHINAVALLI, MCA., M.Phil., Ph.D.**, Academic Director, Department of Computer Applications, for her guidance and encouragement during the course of our study and in successfully accomplishing this project.